

Day 2 Theory: The Terminal, Node.js and JavaScript

Workshop: Web Technology, Hands-on Full-Stack Workshop

Audience: BCA Second Year, Mandsaur University

Time for this day: 2 hours in class, plus about 20 minutes of reading before class

How to use this document

Read it once before class. Do not worry if some parts feel new. Write down your questions and bring them to class.

Yesterday we built the **look** of the Notes Board with HTML and CSS. Today we install our tools, meet the terminal, and learn **JavaScript**, which makes the page do things.

1. Recap of Day 1

- A browser sends a **request**, and a server sends a **response**.
 - **Frontend** is what the user sees. **Backend** is the logic on the server. **Database** stores data.
 - **HTML** is structure, **CSS** is style, and **JavaScript** is behaviour.
-

2. The terminal (command line)

What is it?

A **terminal** is a window where you control the computer by typing **commands**. Instead of clicking a folder, you type its name. Instead of double-clicking a program, you type the program's name.

The terminal is the same as the **CLI** (Command Line Interface) that we met yesterday.

Why do developers use it?

- It is fast. One line can do the work of many clicks.
- Many developer tools, including Node.js and Git, are used from the terminal.
- It works the same way on a laptop, on a server, and on the internet.

Which terminal do we use?

Windows has two common terminals: **Command Prompt** (also called `cmd`) and **PowerShell**. In this workshop we use **Command Prompt** inside VS Code. PowerShell can block some tools with a security message, so Command Prompt is easier for beginners.

Open the terminal in VS Code

1. In VS Code, click **Terminal** in the top menu, then **New Terminal**.
2. A panel opens at the bottom.
3. Next to the **+** icon there is a small down arrow. Click it and choose **Command Prompt**.

Reading the prompt

```
C:\Users\Asha\Documents\Workshop>
```

- This text is called the **prompt**. It shows where you are, which is called the **current folder**.
- You type your command after the `>` sign and press **Enter**.

Commands you will use

Command	What it does	Example
<code>cd</code>	Shows or changes the current folder	<code>cd notes-board</code>
<code>cd ..</code>	Goes one folder up	<code>cd ..</code>
<code>dir</code>	Lists the files and folders in the current folder	<code>dir</code>
<code>mkdir</code>	Makes a new folder	<code>mkdir practice</code>
<code>cls</code>	Clears the screen	<code>cls</code>
<code>echo</code>	Prints text	<code>echo Hello</code>
<code>type</code>	Shows the content of a text file	<code>type hello.txt</code>

Tips that save time

- Press **Tab** to finish a file or folder name for you.
- Press the **Up arrow** to bring back your last command.
- Press **Ctrl + C** to stop a program that is running.
- Commands are not case sensitive on Windows, but we write names in lowercase.
- If a name has spaces, put it inside quotes. This is one reason we avoid spaces in names.

Paths

A **path** is the address of a file or folder.

- **Absolute path:** the full address from the drive, for example `C:\Users\Asha\Documents\Workshop`.
- **Relative path:** the address from where you are now, for example `notes-board\css`.

3. Installing software

An **installer** copies a program to your computer and tells Windows about it. When you install Node.js and Git, the installer also adds them to the **PATH**.

What is PATH?

PATH is a list of folders. When you type a command like `node`, Windows looks for a program with that name in the folders of the list. If Node.js is not in the list, you see:

```
'node' is not recognized as an internal or external command
```

Important: A terminal reads the PATH list only when it **starts**. If you install Node.js while a terminal is open, that terminal does not know about it. Close the terminal, and also close and reopen VS Code, then try again.

What we install today

Tool	What it is	Why we need it
Node.js 24 LTS	A program that runs JavaScript outside the browser	Our backend will be written in JavaScript
npm	A tool that comes with Node.js. It downloads ready-made code packages	We use it from Day 3
Git for Windows	A tool that tracks changes in your code	We use it on Days 4 and 5

What does LTS mean? LTS means **Long Term Support**. It is a version that is kept stable and gets fixes for a long time. It is the right choice for learning and for real projects. We use the **24** line of Node.js so that everyone in the room has the same version.

Versions matter

Different versions of a tool can behave differently. Everyone in the class uses the same installer from the workshop folder, so that we all see the same results.

4. What is Node.js?

JavaScript was first made to run only inside the browser. **Node.js** is a program that lets JavaScript run **outside** the browser, for example on a server. Because of Node.js, we can use **one language** for the frontend and the backend.

You can check that Node.js works from the terminal:

```
node -v
```

It prints the version, for example `v24.x.x`.

You can also run a JavaScript file:

```
node hello.js
```

5. JavaScript basics

JavaScript is a programming language. A **program** is a list of instructions. The computer follows them one by one, from top to bottom.

Where can we see output?

- In the browser, press **F12** and open the **Console** tab.
- In Node.js, output appears in the terminal.

Use `console.log()` to print something:

```
console.log("Hello, world");
```

Comments

A **comment** is a note for humans. The computer ignores it.

```
// This is a one-line comment
```

Variables

A **variable** is a named box that stores a value.

```
const college = "Mandsaur University"; // cannot be changed later
let count = 0;                          // can be changed later
count = count + 1;
```

- Use `const` by default.
- Use `let` only when the value must change.

Data types

Type	Example	Meaning
String	"Hello"	Text, written inside quotes
Number	42 or 3.5	Numbers
Boolean	true or false	Yes or no
Undefined	undefined	A variable that has no value yet
Null	null	An empty value on purpose

Operators

```
const a = 10;
const b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1 (the remainder)
console.log(a === b); // false (is it exactly equal?)
console.log("Hi " + "there"); // Hi there (joining text)
```

Always use `===` to compare. Do not use a single `=` for comparing. A single `=` **stores** a value.

Conditions

A **condition** lets the program choose.

```
const marks = 62;

if (marks >= 40) {
  console.log("Pass");
} else {
  console.log("Fail");
}
```

Arrays

An **array** is a list of values. Our notes are stored in an array.

```
const days = ["Mon", "Tue", "Wed"];

console.log(days[0]);      // Mon (counting starts at 0)
console.log(days.length);  // 3

days.push("Thu");         // add at the end
days.unshift("Sun");      // add at the start
```

Objects

An **object** groups related values. Each value has a name, called a **key**.

```
const note = {
  id: 1,
  title: "Welcome",
  text: "This is my first note."
};

console.log(note.title);    // Welcome
```

An array of objects is the most common way to store a list of things, such as notes.

```
const notes = [  
  { id: 1, title: "Welcome", text: "Hello" },  
  { id: 2, title: "Study plan", text: "Learn JavaScript" }  
];
```

Loops

A **loop** repeats work.

```
for (const note of notes) {  
  console.log(note.title);  
}
```

Functions

A **function** is a named block of code that you can run again and again.

```
function greet(name) {  
  return "Hello, " + name;  
}  
  
console.log(greet("Asha")); // Hello, Asha
```

- `name` is a **parameter**: the input.
- `return` gives back the **result**.
- `greet("Asha")` is a **function call**.

6. JavaScript in the browser: the DOM

When the browser opens an HTML page, it builds a **tree** of objects in memory, one for every element. This tree is called the **DOM** (Document Object Model). JavaScript can read and change the DOM. When it changes the DOM, the page on the screen changes immediately, with no reload.

```
document
  html
    head
    body
      header
      main
        form
        div#notes-list
          article.note
```

Finding an element

```
const list = document.getElementById("notes-list");
```

Changing text

```
list.textContent = "No notes yet.";
```

Creating and adding elements

```
const article = document.createElement("article");
article.className = "note";

const heading = document.createElement("h3");
heading.textContent = "Welcome";

article.appendChild(heading); // put the h3 inside the article
list.appendChild(article);    // put the article inside the list
```

Events

An **event** is something that happens on the page: a click, a key press, a form submit. We tell the browser which function to run when an event happens.

```
button.addEventListener("click", function () {
  console.log("The button was clicked");
});
```

The page and the script

To load JavaScript into a page, we add a script tag at the **end of the body**, so that the page elements already exist when the script runs:

```
<script src="js/app.js"></script>
</body>
```

7. Our Notes Board code, explained

Today we make the Notes Board work in the browser. The code has nine small parts.

1. **Data:** an array named `notes` that holds the notes. Each note is an object with `id`, `title` and `text`.
2. **Find the parts of the page:** we use `getElementById` to get the form, the inputs and the list.
3. **showMessage** : shows a short message under the form.
4. **renderNotes** : clears the list and draws every note from the array. It also shows the count.
5. **addNote** : adds a new note at the start of the array and draws again.
6. **deleteNote** : finds a note by its `id`, removes it from the array and draws again.
7. **The submit event:** when the form is submitted, we stop the page reload, check the input, and call `addNote`.
8. **The input event:** shows how many letters were typed.
9. **First draw:** `renderNotes()` runs once when the page opens.

This idea is used in almost every app: **keep the data in one place, and draw the page from the data.**

Why do the notes disappear when I refresh?

The `notes` array lives in the **memory of the browser**. When you refresh, the browser starts again from the beginning, and the array is rebuilt from the code. Your new notes were never saved anywhere.

To keep data, we need a place outside the page: a **server** with a **database**. That is our plan for Days 3 and 4.

8. A first safety lesson: never trust user input

Anyone can type anything into our form, including HTML code such as `<img src=x onerror=alert(1)>`.

If we put user text into the page as **HTML**, the browser may treat it as real code and run it. This attack is called **XSS** (cross-site scripting). It is one of the most common security problems on the web.

Our code is safe because it uses `textContent`. `textContent` treats the text as plain text, so any HTML code is shown as letters and never runs.

```
heading.textContent = note.title;    // SAFE
// heading.innerHTML = note.title;    // UNSAFE with user input. Do not do this.
```

The same idea is used for links on the About page: we only allow links that start with `https://`.

Remember this rule for your whole career: **user input is never trusted**.

9. Server basics: localhost and ports

At the end of today's lab we run our first small server.

- **localhost** is a special name that means "this same computer". A server running on your computer can be opened with `http://localhost`.
- A **port** is a numbered door on a computer. One computer can run many servers, and each one listens on a different port. We use port **3000**. The address is `http://localhost:3000`.
- When the server runs, the terminal is busy. To stop it, press **Ctrl + C**.

Our first server answers every request with the same text. The function we give to the server runs once for every request, and it sends a response. This is exactly the request and response idea from Day 1, now written in code.

```
const http = require("node:http");

const server = http.createServer(function (request, response) {
  response.end("Hello from my first Node.js server!");
});

server.listen(3000);
```

On Day 3 we replace this by **Express**, a tool that makes servers much easier to write.

10. Quick revision

1. The **terminal** is where we type commands. We use **Command Prompt**.
 2. **PATH** is the list of folders where Windows looks for commands. Reopen the terminal after installing.
 3. **Node.js** runs JavaScript outside the browser. We use the **24 LTS** version.
 4. In JavaScript: **variables, arrays, objects, functions, events**.
 5. The **DOM** is the page in memory. JavaScript changes it, and the screen changes.
 6. Data kept only in the browser is lost on refresh. We need a server and a database.
 7. Always use `textContent` for user text. User input is never trusted.
-

11. Homework for Day 3

1. If you have not yet created your **GitHub account**, do it at home. Write down your username.
 2. Try the **extra challenges** at the end of the Day 2 Lab Sheet.
 3. Run `node setup-check.js` again at home or in the lab. All lines should show **PASS**.
 4. Write down at least two questions for Day 3.
-

Glossary

Word	Meaning
Array	A list of values
Command Prompt	A terminal program in Windows
Console	The place where <code>console.log</code> prints (browser or terminal)
DOM	The page in memory, which JavaScript can change
Event	Something that happens on the page, such as a click
Function	A named block of code that can be run again
Installer	A program that puts another program on your computer
LTS	Long Term Support: a stable version with fixes for a long time
localhost	A name that means "this same computer"
Node.js	A program that runs JavaScript outside the browser
npm	A tool that comes with Node.js and downloads ready-made code packages
Object	A group of related values with names
PATH	The list of folders where Windows looks for commands
Port	A numbered door that a server listens on
Prompt	The text that shows where you are in the terminal
Variable	A named box that stores a value
XSS	An attack that runs harmful code by using unsafe user input